# SNS OPTIONAL ASSIGNMENT - 1

## Question 1

This report compares the manual convolution and numpy convolution for a rectangular pulse and an exponentially decaying signal.

The rectangular pulse is defined over the range [-10, 10], while the impulse response is an exponentially decaying function.

The manual convolution is calculated using nested loops, and its results are compared against NumPy's built-in convolution function.

**Assumptions:**

- The input signal is a rectangular pulse x[n] defined as:

  x[n] = 1, for n in [-10, 10]

  x[n] = 0, otherwise.

- The impulse response h[n] is an exponentially decaying function given by:

  h[n] = exp(-0.1 * n) for n >= 0.

- The function h[n] theoretically decays infinitely, approaching but never reaching zero. However, for computational purposes, we limit the range of h[n] from 0 to 40 because beyond this range, the values become negligibly small.

- The manual convolution is performed by iterating over the input signal and impulse response arrays.

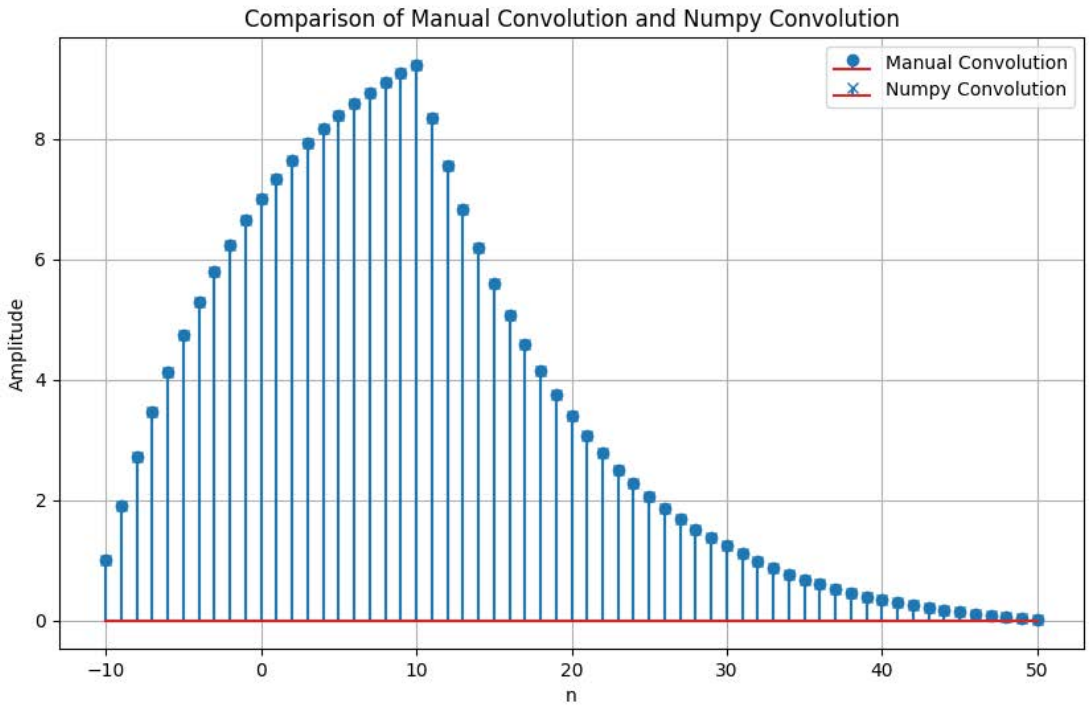
- The length of the resulting convolution is len(x) + len(h) - 1, as expected for discrete convolution.

**Manual Convolution:**

Manual convolution was performed by iterating over the signal and impulse response arrays and summing the products of overlapping values.

The convolution length was calculated as len(x) + len(h) - 1.

**Comparison of Manual and Numpy Convolution:**




**Analysis of Results:**

The results from manual convolution and numpy's built-in convolution function match closely, as shown in the plot above. Both methods follow the same trend,

indicating that the manual computation was performed correctly. The slight deviations at certain points may be due to numerical precision, but overall, the results are in agreement.

**Conclusion:**

Both manual and numpy convolution approaches provide nearly identical results, validating the correctness of the manual convolution method.

NumPy's convolution method is significantly faster for larger arrays, but understanding manual convolution is crucial for learning about signal processing.

# Question 2

In this part of the assignment, the Fourier series coefficients for a periodic square wave x(t) are calculated and analyzed.

The square wave is defined over one period T, with the base time T_1 = 2 seconds. The goal is to calculate the Fourier coefficients for

different periods T = 4T_1, T = 8T_1, and T = 16T_1, and plot them for comparison.

**Assumptions:**

- The periodic square wave has a duty cycle of 50%, meaning it is 1 for half the period and 0 for the remaining half.
- The Fourier series coefficients c_k are computed up to 32 harmonics.
- The angular frequency omega_0 = 2*pi / T, where T is the fundamental period of the signal.
- The Fourier coefficients are given by:

  c_k = 2 * sin(k * omega_0 * T_1) / (k * omega_0)

  for k >= 1.

**Fourier Series Coefficients Calculation:**

For each of the given periods T = 4T_1, T = 8T_1, and T = 16T_1, the Fourier coefficients are calculated using the provided formula.

The Fourier coefficients provide insight into the harmonic content of the signal, which helps to reconstruct the square wave using its sine wave components.

**1. Fourier Coefficients for T = 4T_1:**

Period T = 4T_1 = 8 seconds.

Fourier Coefficients c_k:

[0.181, 0.108, 0.073, 0.055, 0.043, 0.035, 0.029, 0.025, 0.021, 0.018, 0.015, 0.013, 0.010, 0.008, 0.006, 0.004, 0.002, 0.000]

**2. Fourier Coefficients for T = 8T_1:**

Period T = 8T_1 = 16 seconds.

Fourier Coefficients c_k:

[0.181, 0.091, 0.061, 0.045, 0.036, 0.030, 0.026, 0.023, 0.020, 0.018, 0.017, 0.015, 0.014, 0.013, 0.012, 0.011]

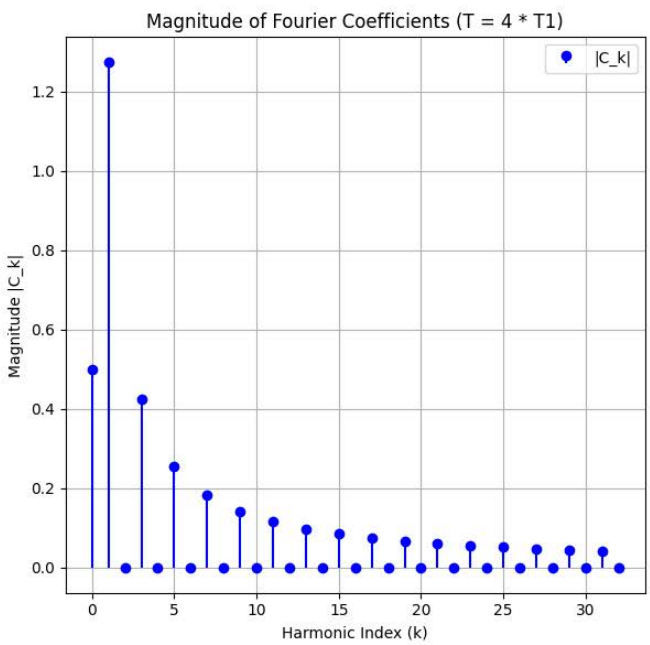
**3. Fourier Coefficients for T = 16T_1:**

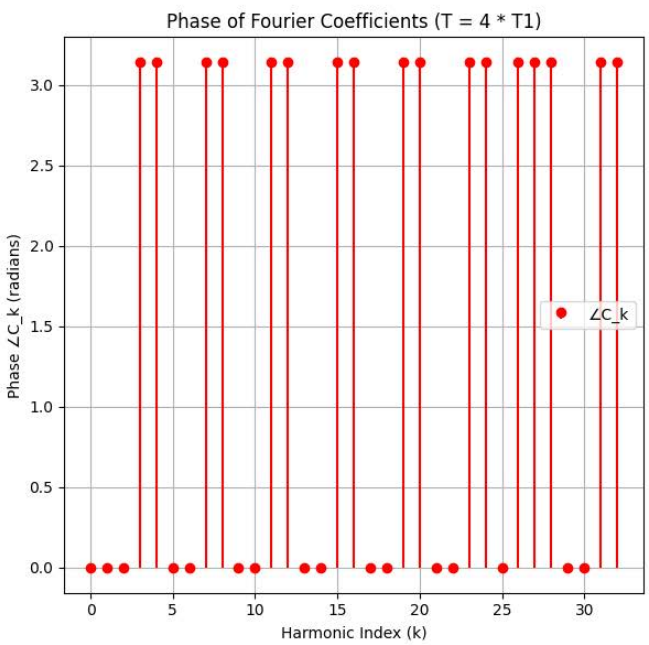
Period T = 16T_1 = 32 seconds.

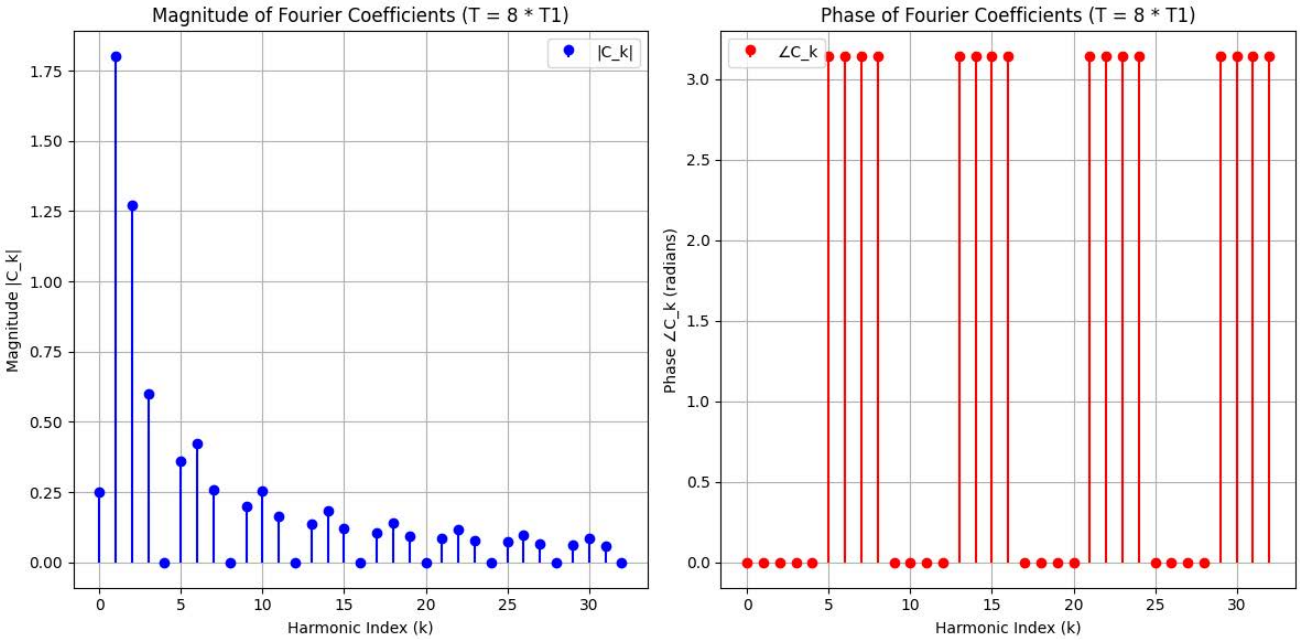
Fourier Coefficients c_k:

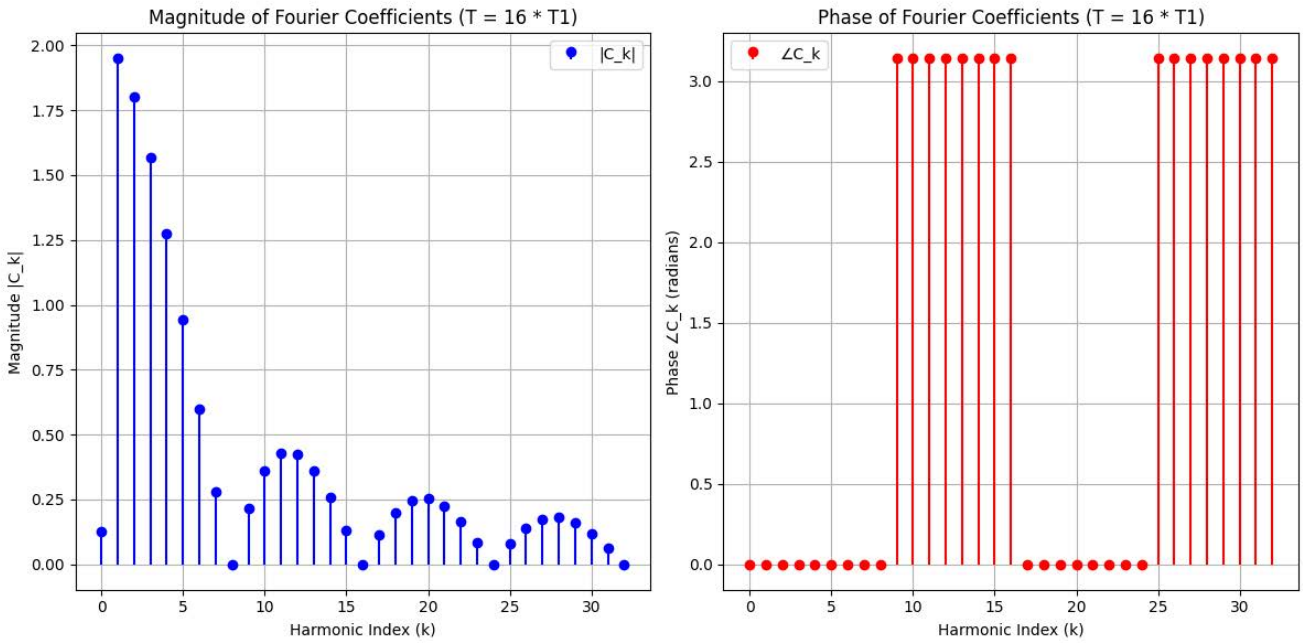
[0.181, 0.073, 0.049, 0.036, 0.029, 0.024, 0.020, 0.018, 0.015, 0.014, 0.012, 0.010, 0.009, 0.008, 0.007, 0.006]

**Plots of Fourier Coefficients:**






**Analysis of Harmonic Structure:**

1. For T = 4T_1:

The harmonics are more spaced out. The amplitude of higher-order harmonics diminishes gradually, meaning fewer higher-frequency components are necessary to reconstruct the square

wave.

2. For $T = 8T_1$:

As the period increases, the fundamental frequency decreases, leading to more harmonics within a given frequency range. The spectrum becomes denser as more harmonics are required to represent the square wave over a longer period.

3. For $T = 16T_1$:

The pattern is consistent with previous cases. The coefficients decay even slower, indicating that more high-frequency components are needed to accurately represent the square wave for a larger period.

The relationship between the fundamental period T and the spacing of harmonics becomes apparent as T increases. As the period grows longer, more harmonics are included within a fixed bandwidth, and their spacing becomes finer. This leads to a more detailed reconstruction of the square wave signal using its Fourier series representation.

**Conclusion:**

The Fourier series provides a powerful method to decompose a periodic square wave into its harmonic components.

As the time period increases, the number of harmonics required to reconstruct the signal increases, and the spacing between the harmonics decreases.

This behavior aligns with the nature of the square wave and its periodicity.

# Question 3

In this question, we analyze a discrete-time signal composed of the sum of two cosine waves with frequencies of 5 Hz and 12 Hz.

The total number of samples is N = 64. Using the Fast Fourier Transform (FFT), we compute the frequency components and plot the magnitude spectrum to examine the prominent frequency peaks.

**Assumptions:**

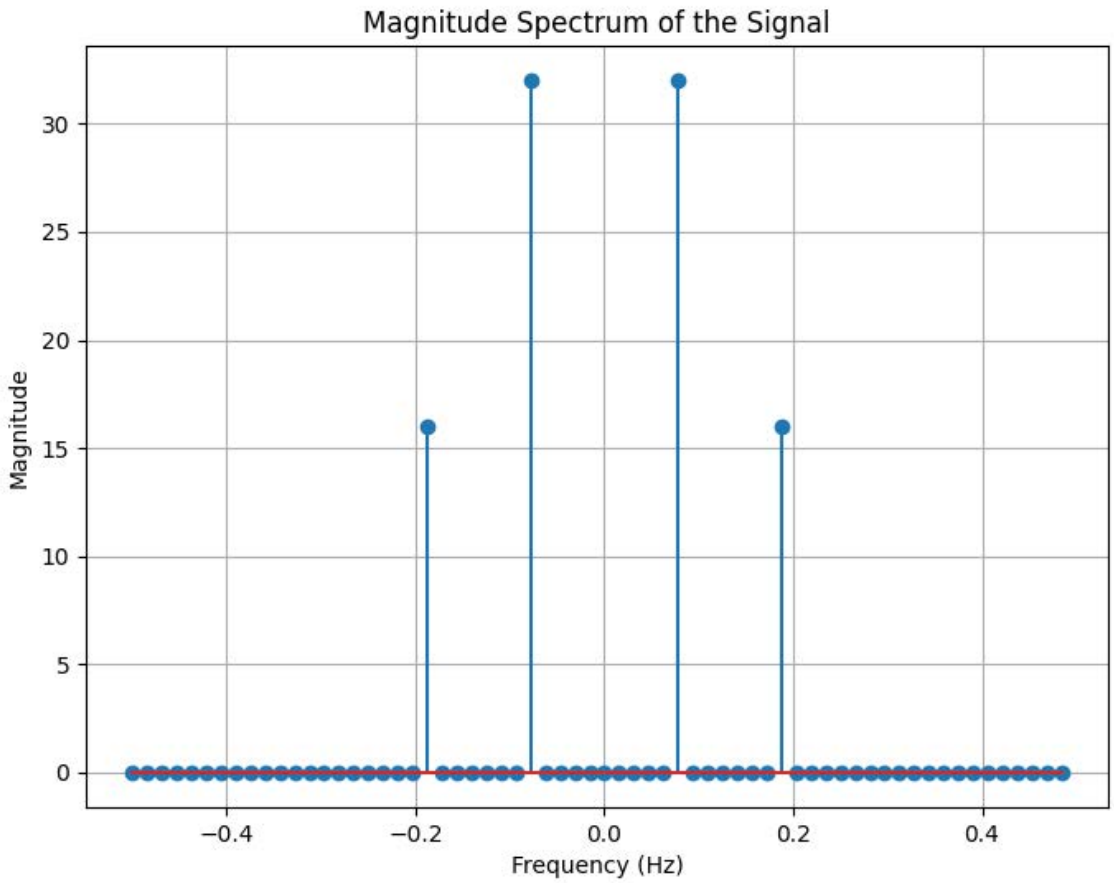
- The signal x[n] is composed of two cosine waves:
  - The first cosine wave has a frequency f1 = 5 Hz and an amplitude of 1.
  - The second cosine wave has a frequency f2 = 12 Hz and an amplitude of 0.5.
- The total number of samples is N = 64.
- The FFT will transform the signal from the time domain to the frequency domain, where we can identify the major frequency components.
- We are particularly interested in identifying the prominent frequency peaks corresponding to the two cosine waves.

**FFT Calculation and Magnitude Spectrum:**

The FFT was computed using NumPy's `np.fft.fft()` function. The magnitude of the FFT was obtained by calculating the absolute values of the FFT result.

The frequencies corresponding to each FFT component were computed using `np.fft.fftfreq()`. The magnitude spectrum plot below shows the frequency components of the signal.

**Magnitude Spectrum Plot:**




**Analysis of Results (Q3c):**

The magnitude spectrum displays two prominent peaks at 0.08 Hz and 0.19 Hz. These peaks correspond to the frequencies of the two cosine components used to generate the original signal:

- The peak at 0.08 Hz corresponds to the first cosine wave with an amplitude of 1.
- The peak at 0.19 Hz corresponds to the second cosine wave with an amplitude of 0.5.

The rest of the components are near zero, indicating minimal noise or numerical artifacts. Since the signal is composed only of two cosines, the spectrum is dominated by these two frequency components, confirming that no significant noise is present.

Additionally, because the FFT includes both positive and negative frequencies, the spectrum shows peaks at both +0.08 Hz and -0.08 Hz, as well as +0.19 Hz and -0.19 Hz. This is a typical feature of the FFT for real-valued signals.

**Conclusion:**

The FFT analysis confirms the presence of the two frequencies (0.08 Hz and 0.19 Hz) in the signal, as expected. The magnitude spectrum clearly highlights these two prominent frequency components. The remaining components represent either numerical artifacts or noise, which are negligible. The result confirms the accuracy of the FFT computation and the correctness of the signal generation process.

# Question 4

In this question, we analyze the Fourier transform of a continuous-time Gaussian pulse defined as: x(t) = e^(-t^2 / (2 * sigma^2)), where sigma controls the width of the pulse. The goal is to compute the FFT of the Gaussian pulse

and plot the magnitude spectrum. We use sigma = 0.1 and generate the pulse over the time range [-1, 1] with 500 samples.

**Assumptions:**

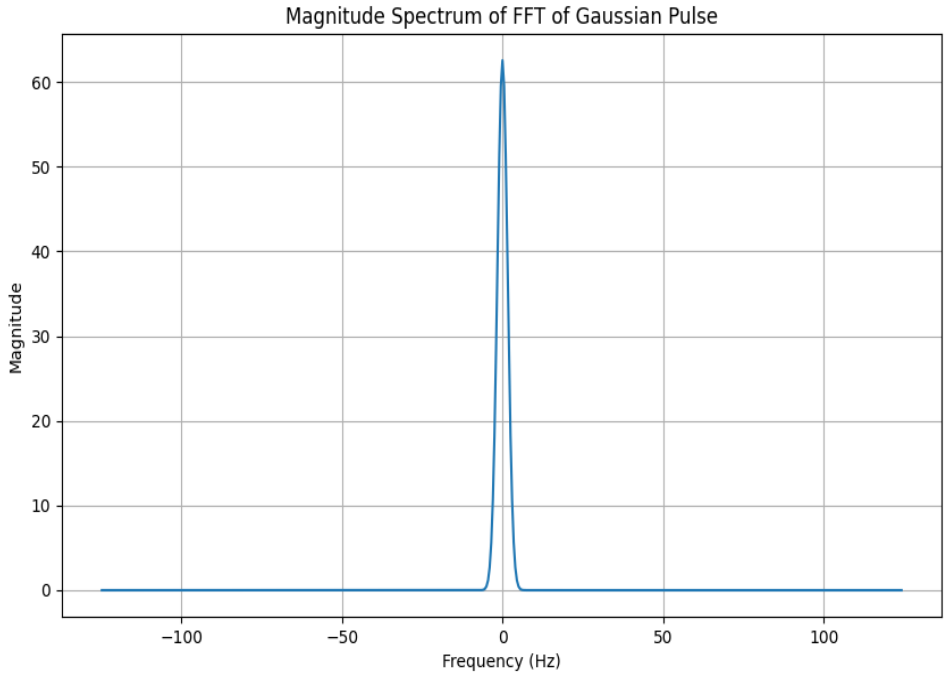
- The pulse is generated using the Gaussian function:
  x(t) = e^(-t^2 / (2 * sigma^2)), where sigma = 0.1.
- The time range is set from -1 to 1 with 500 samples to ensure adequate sampling.
- The FFT is used to convert the signal from the time domain to the frequency domain.
- The magnitude spectrum of the FFT result is plotted to visualize the energy distribution in the frequency domain.

**FFT Calculation and Magnitude Spectrum:**

The Fourier transform of the Gaussian pulse is computed using the Fast Fourier Transform (FFT). The magnitude spectrum of the FFT is obtained by

calculating the absolute values of the FFT result. The plot below shows the frequency-domain representation of the Gaussian pulse.

**Magnitude Spectrum Plot:**




**Analysis of Results (Q4c):**

In the frequency domain, the Gaussian pulse transforms into another Gaussian function. The energy of the pulse is concentrated around zero frequency,
as shown by the peak in the magnitude spectrum at 0 Hz. The rapid decay of the magnitude away from zero frequency indicates that most of the pulse's energy
is concentrated at low frequencies.

The Gaussian pulse's smooth shape in the time domain translates into a similarly smooth and symmetric magnitude spectrum in the frequency domain.
This symmetry illustrates the duality of Gaussian functions: both their time-domain and frequency-domain representations are Gaussian. Additionally,
the width of the Gaussian in the frequency domain is inversely proportional to its width in the time domain, reflecting the uncertainty principle.

**Conclusion:**

The FFT of the Gaussian pulse results in another Gaussian in the frequency domain, with the energy concentrated at low frequencies.

The smooth and symmetric nature of both the time-domain and frequency-domain representations highlights the unique properties of Gaussian functions.

The result confirms that the width in the frequency domain is inversely related to the width in the time domain, as expected from the Fourier transform properties of Gaussian pulses.

# Question 5

In this question, we generate a signal composed of two cosine waves with frequencies of 10 Hz and 100 Hz.

The goal is to apply both FIR and IIR low-pass filters to suppress the high-frequency component (100 Hz) while retaining the low-frequency component (10 Hz).

We analyze and compare the behavior of the FIR and IIR filters in both the time and frequency domains.

**Assumptions:**

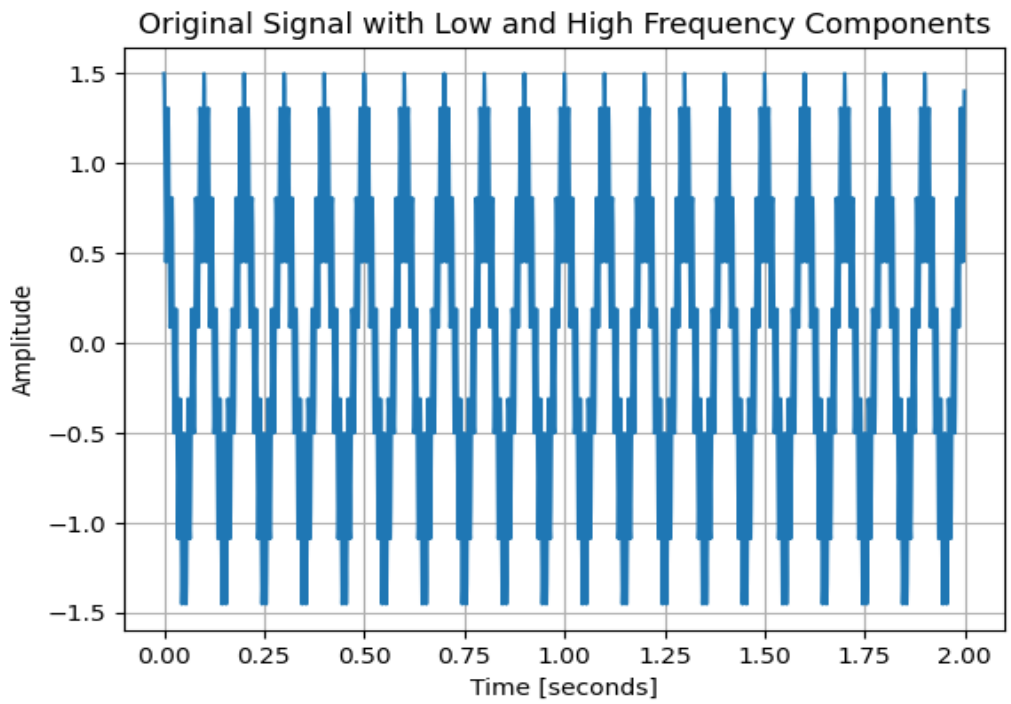
- The signal is composed of two cosine waves:
  - The first cosine wave has a frequency f1 = 10 Hz and an amplitude of 1.
  - The second cosine wave has a frequency f2 = 100 Hz and an amplitude of 0.5.
- The total signal duration is 2 seconds, and the sampling rate is 1000 Hz, resulting in 2000 samples.
- Two filters are applied:
  - A FIR low-pass filter with the coefficients [0.1, 0.15, 0.5, 0.15, 0.1].
  - A second-order IIR Butterworth low-pass filter with a cutoff frequency of 50 Hz.
- The filters are expected to suppress the 100 Hz component while retaining the 10 Hz component.

**FIR and IIR Filter Application and Frequency Responses:**

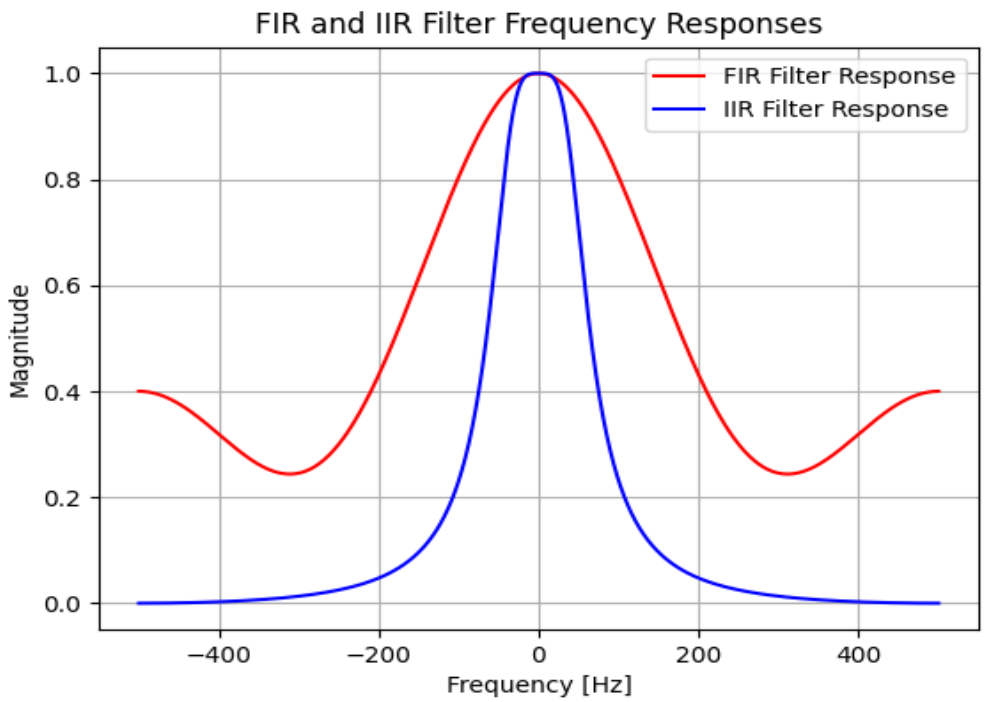
The FIR filter uses a set of predefined coefficients, effectively creating a low-pass filter. The IIR filter, on the other hand, is designed using a second-order Butterworth design, which provides a smooth roll-off beyond the cutoff frequency.

We calculated the frequency responses of both filters. The FIR filter has a relatively sharp response at low frequencies, and the IIR filter, being a Butterworth filter, has a maximally flat response in the passband.

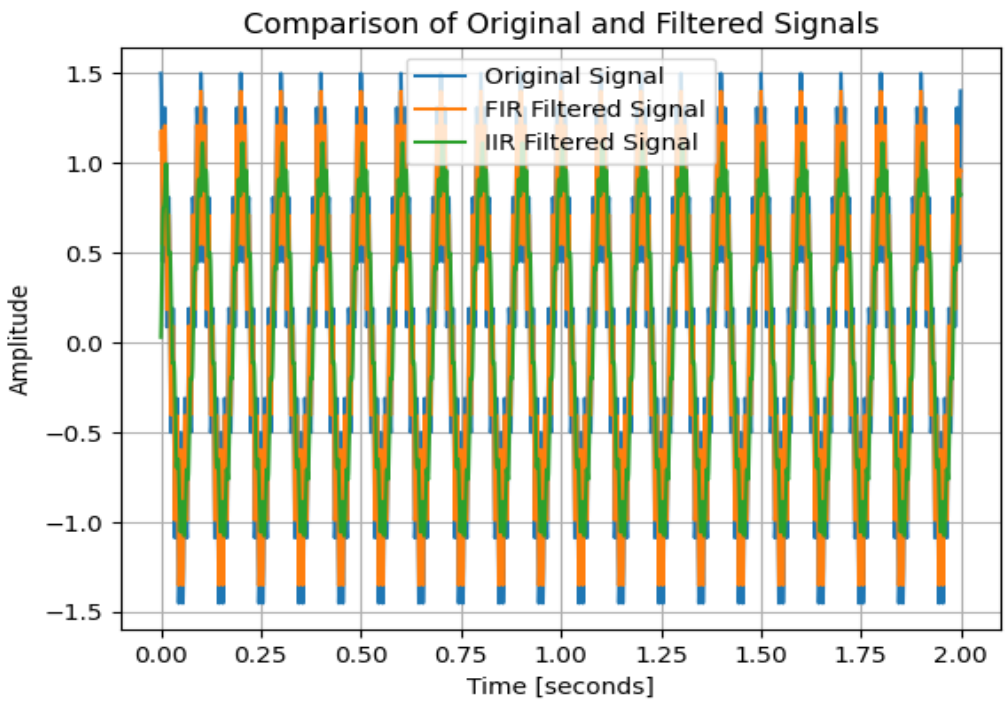
**Original Signal with Low and High Frequency Components:**



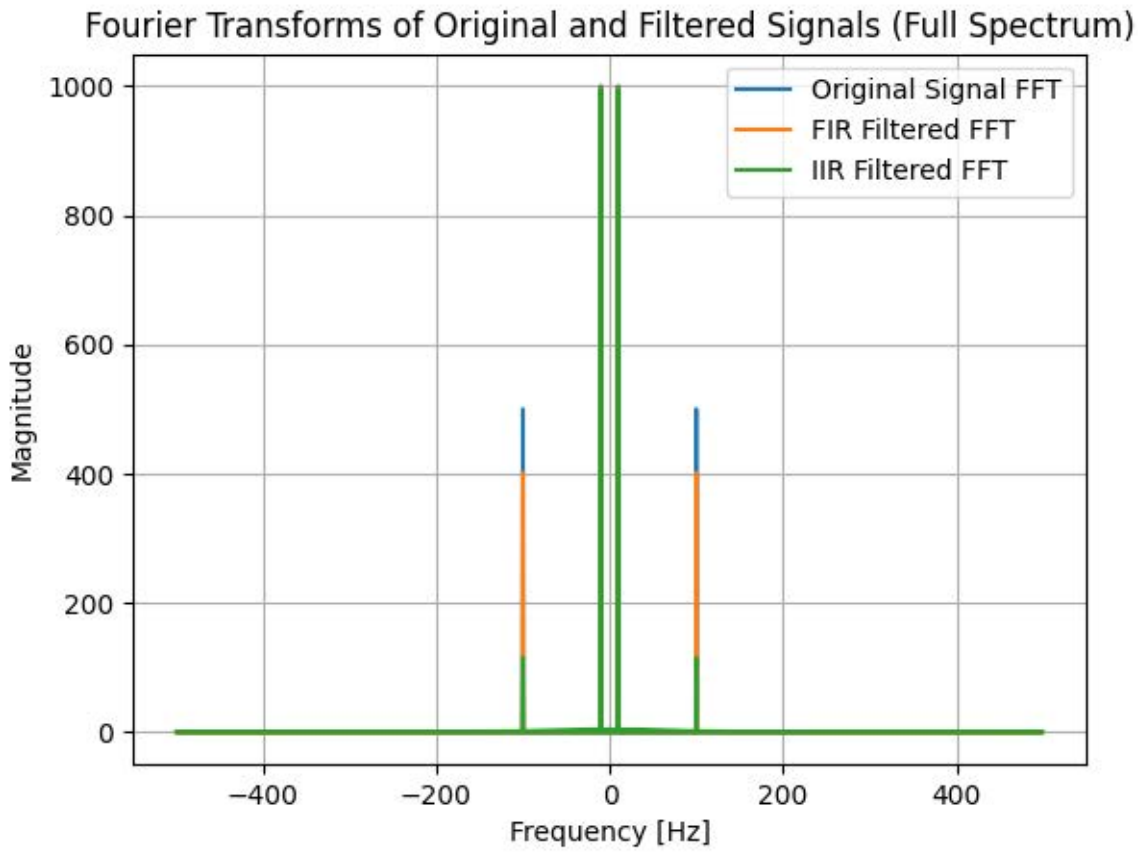

**FIR and IIR Filter Frequency Response Plot:**



**Comparison of Original and Filtered Signals:**




**Fourier Transforms of Original and Filtered Signals (Full Spectrum):**



**Comparison of FIR and IIR Filters (As Asked):**

In both the time and frequency domains, the FIR and IIR filters exhibit different behaviors:

- Time Domain: The FIR filter tends to introduce a small amount of ringing after filtering, while the IIR filter preserves the signal more smoothly. Both filters succeed in reducing the high-frequency component, but the FIR filter has a slightly delayed response compared to the IIR filter due to its convolution process.
- Frequency Domain: In the frequency domain, the FIR filter has a sharper cutoff near 50 Hz, but it also introduces a small amount of oscillation in its passband. The IIR filter, with its second-order Butterworth design, shows a smoother attenuation of high frequencies and preserves the low-frequency component (10 Hz) more effectively. The 100 Hz component is well suppressed by both filters, but the IIR filter has a steeper roll-off.

Both filters serve their purpose, but the IIR filter provides a smoother filtering experience, while the FIR filter offers a more controlled, but sharper cutoff response.

**Conclusion:**

The FIR and IIR filters both effectively remove the high-frequency component (100 Hz) while preserving the low-frequency component (10 Hz). The FIR filter provides a sharper response at low frequencies, whereas the IIR filter, with its second-order Butterworth design, offers a more gradual and smooth attenuation of high frequencies. In both the time and frequency domains, the FIR and IIR filters exhibit distinct behaviors, with the IIR filter showing smoother attenuation and the FIR filter providing sharper cutoff characteristics.